

Laboratório 5

Relatório

Nícolas Rosenthal Dal Corso

Ciência de Dados (INF 01090/2026)
May, 2026

Sumário

1	Visão Geral	2
2	Dados e Pré-processamento	2
2.1	Conjunto de Dados	2
2.2	Remoção de Outliers	2
2.3	Transformação da Target	3
2.4	Correção de Tipos	3
2.5	Imputação de Valores Ausentes	3
2.6	Encoding Ordinal	4
3	Engenharia de Features	4
3.1	Área Total e Contagens	4
3.2	Features Temporais	4
3.3	Flags Binárias	4
3.4	Interações Multiplicativas	5
3.5	Log-Transforms de Features Assimétricas	5
3.6	Razões (<i>Ratios</i>)	5
3.7	Scores Compostos	5
3.8	Sazonalidade de Venda	5
4	Arquitetura do Modelo	6
4.1	Nível 1 Base Learners	6
4.2	Busca de Hiperparâmetros com Optuna	6

4.3	Validação Cruzada (10-fold, 3 repetições)	6
4.4	Nível 2 Meta-Learners	7
4.5	Meta-Blending Final (Optuna)	7
4.6	Pseudo-Labeling (2ª Passagem)	7
5	Pré-processamento Inside-Fold	8
6	Resultados	8
6.1	RMSE por Base Learner (OOF, escala logarítmica)	8
6.2	Resultado Final	9
7	Conclusão	9

1 Visão Geral

Este relatório descreve a implementação do *pipeline* de predição de preços de imóveis desenvolvido para a competição *House Prices - Advanced Regression Techniques* (Kaggle), adaptada para o conjunto de dados de avaliação da disciplina INF01090/2026.

O modelo final é um **stack ensemble** de dois níveis com meta-blending via Optuna, cujo OOF RMSE (em escala log) obtido foi de **0.112577**.

Todas as submissões ocorreram sob o nome **n-rosenthal**.

2 Dados e Pré-processamento

2.1 Conjunto de Dados

Atributo	Valor
Linhas (treino)	1 016 (após remoção de outliers)
Linhas (teste)	438
Features brutas	80 variáveis
Features finais	125 (após engenharia)
Target	SalePrice (numérica, contínua)

2.2 Remoção de Outliers

Cinco regras de filtragem foram aplicadas sobre o conjunto de treino:

```
train_df = train_df[train_df["GrLivArea"] < 4500]
train_df = train_df[~((train_df["GrLivArea"] > 4000) & (train_df["SalePrice"] < 200
```

```
train_df = train_df[train_df["LotArea"] < 100_000]
train_df = train_df[train_df["TotalBsmtSF"] < 6_000]
train_df = train_df[~((train_df["OverallQual"] <= 4) & (train_df["SalePrice"] > 300_000))]
```

Essas regras removem imóveis com *living area* muito grande, lotes exageradamente grandes e combinações de baixa qualidade com preço alto, que distorceriam o aprendizado.

2.3 Transformação da Target

O preço de venda (`SalePrice`) foi transformado via `log1p` antes do treinamento e revertido via `expm1` nas predições finais. Isso reduz a assimetria da distribuição e estabiliza o treinamento dos modelos de boosting.

2.4 Correção de Tipos

`MSSubClass`, `MoSold` e `YrSold` são variáveis **nominais**, não números contínuos. A conversão para `string` é feita **antes** da separação entre `cat_cols` e `num_cols`, garantindo que sejam tratadas como categorias pelo `TargetEncoder` e não como grandezas ordenadas.

```
def fix_dtypes(df):
    for col in ["MSSubClass", "MoSold", "YrSold"]:
        if col in df.columns:
            df[col] = df[col].astype(str)
    return df
```

2.5 Imputação de Valores Ausentes

Duas estratégias foram adotadas:

- **Colunas categóricas sem valor** (`Alley`, `MasVnrType`, `MiscFeature`): preenchidas com a string `"None"`, que é mapeada para 0 nos encodings ordinais.
- **Colunas numéricas** (`MasVnrArea`, `BsmtFinSF1/2`, `GarageArea`, etc.): preenchidas com 0, representando ausência do atributo.
- **GarageYrBlt**: imputado com `YearBuilt` do próprio imóvel (não com 0, que geraria um ano inválido e introduziria viés semântico).

```
mask = df["GarageYrBlt"].isna()
df.loc[mask, "GarageYrBlt"] = df.loc[mask, "YearBuilt"]
```

2.6 Encoding Ordinal

Variáveis de qualidade com escala natural (Ex > Gd > TA > Fa > Po) foram codificadas numericamente por mapeamento direto, evitando que o modelo trate categorias ordenadas como nominais:

Coluna	Escala
ExterQual	Ex=5, Gd=4, TA=3, Fa=2, Po=1, NA=0
BsmtQual	(idem acima)
KitchenQual	(idem acima)
BsmtFinType1	GLQ=6, ALQ=5, Unf=1, NA=0
BsmtExposure	Gd=4, Av=3, Mn=2, No=1, NA=0
Functional	Typ=8, Min1=7, Sal=1
GarageFinish	Fin=3, RFn=2, Unf=1, NA=0

3 Engenharia de Features

Foram criadas 45 features adicionais, organizadas em sete grupos:

3.1 Área Total e Contagens

```
df["TotalSF"] = df["TotalBsmtSF"] + df["1stFlrSF"] + df["2ndFlrSF"]
df["TotalBathrooms"] = df["FullBath"] + 0.5*df["HalfBath"] + \
    df["BsmtFullBath"] + 0.5*df["BsmtHalfBath"]
df["TotalPorchSF"] = df["OpenPorchSF"] + df["EnclosedPorch"] + \
    df["3SsnPorch"] + df["ScreenPorch"]
df["GarageScore"] = df["GarageCars"] * df["GarageArea"]
```

3.2 Features Temporais

```
df["HouseAge"] = df["YrSold"].astype(int) - df["YearBuilt"]
df["RemodAge"] = df["YrSold"].astype(int) - df["YearRemodAdd"]
df["TimeSinceRemod"] = df["YrSold"].astype(int) - df["YearRemodAdd"]
df["IsNew"] = (df["YrSold"].astype(int) == df["YearBuilt"]).astype(int)
df["IsRemodeled"] = (df["YearRemodAdd"] != df["YearBuilt"]).astype(int)
```

3.3 Flags Binárias

Nove flags indicam presença/ausência de atributos importantes: HasPool, Has2ndFloor, HasGarage, HasBsmt, HasFireplace, IsRemodeled, IsNew, HasMasVnr, HasPorch.

3.4 Interações Multiplicativas

```
df["OverallQual_TotalSF"] = df["OverallQual"] * df["TotalSF"]
df["OverallQual_GrLivArea"] = df["OverallQual"] * df["GrLivArea"]
df["OverallQual_sq"] = df["OverallQual"] ** 2
df["QualCondScore"] = df["OverallQual"] * df["OverallCond"]
df["QualAge"] = df["OverallQual"] / df["HouseAge"].clip(1)
df["NewAndQual"] = df["IsNew"] * df["OverallQual"]
```

3.5 Log-Transforms de Features Assimétricas

```
df["LotAreaLog"] = np.log1p(df["LotArea"])
df["GrLivAreaLog"] = np.log1p(df["GrLivArea"])
df["TotalSFLog"] = np.log1p(df["TotalSF"])
df["GarageAreaLog"] = np.log1p(df["GarageArea"])
```

3.6 Razões (*Ratios*)

```
df["LivAreaRatio"] = df["GrLivArea"] / df["TotalSF"].clip(1)
df["BsmtRatio"] = df["TotalBsmtSF"] / df["TotalSF"].clip(1)
df["GarageRatio"] = df["GarageArea"] / df["GrLivArea"].clip(1)
df["LotFrontRatio"] = df["LotFrontage"].fillna(0) / df["LotArea"].clip(1)
df["BsmtFinRatio"] = df["BsmtFinSF1"] / df["TotalBsmtSF"].clip(1)
df["LotAreaPerRoom"] = df["LotArea"] / df["TotRmsAbvGrd"].clip(1)
df["SFPerRoom"] = df["GrLivArea"] / df["TotRmsAbvGrd"].clip(1)
```

3.7 Scores Compostos

```
df["ExterScore"] = df["ExterQual"] * df["ExterCond"]
df["KitchenScore"] = df["KitchenQual"] * df["GrLivArea"]
df["FireScore"] = df["FireplaceQu"] * df["Fireplaces"]
df["BsmtScore"] = df["BsmtExposure"] * df["BsmtQual"]
df["GarageFullScore"] = df["GarageQual"] * df["GarageCars"]
```

3.8 Sazonalidade de Venda

```
mo = df["MoSold"].astype(int)
df["SaleSeasonQ1"] = mo.isin([1, 2, 3]).astype(int)
# Q2, Q3, Q4 idem
```

4 Arquitetura do Modelo

O *pipeline* adota uma arquitetura de **stack ensemble** em dois níveis.

4.1 Nível 1 Base Learners

Oito modelos de gradient boosting formam o primeiro nível:

ID	Algoritmo	Profundidade	Observações
cat_a	CatBoost	5	Config principal; usa Optuna
cat_b	CatBoost	6	Mais árvores, LR menor
cat_c	CatBoost	4	Máxima regularização
cat_d	CatBoost	7	Lado mais profundo
cat_e	CatBoost (Lossguide)	6	Política de crescimento diferente
lgb_best	LightGBM	5	num_leaves=20; usa Optuna
xgb_best	XGBoost	4	gamma=0.1, conservador
hgb	HistGradBoost (sklearn)		Implementação ortogonal

Os modelos **CatBoost** dominaram o ranking OOF (0.1140.117 RMSE), enquanto LGB, XGB e HGB ficaram na faixa 0.1240.126, contribuindo com diversidade ao ensemble.

4.2 Busca de Hiperparâmetros com Optuna

Os hiperparâmetros de **cat_a** e **lgb_best** são encontrados por busca bayesiana com **Optuna** (**n_trials=50**), usando OOF RMSE em validação cruzada de 5 folds como métrica. O espaço de busca cobre **iterations**, **learning_rate**, **depth**, **l2_leaf_reg**, **bagging_temperature**, **border_count** e **random_strength** (CatBoost), e os parâmetros análogos do LGB. Os parâmetros dos demais modelos (**cat_b** a **cat_e**, **xgb_best**, **hgb**) são fixos e cobrem regiões complementares do espaço de hipóteses, garantindo diversidade.

4.3 Validação Cruzada (10-fold, 3 repetições)

O treinamento dos base learners utiliza **RepeatedKFold** com 10 splits e **3 repetições** (**N_REPEATS=3**, totalizando 30 folds), o que reduz a variância da estimativa OOF em 31,73%.

Para cada fold:

1. **SimpleImputer(median)** + **RobustScaler** aplicados **apenas nos dados de treino**.

2. `TargetEncoder(smoothing=10)` ajustado **apenas nos dados de treino**.
3. `PCA(n_components=10)` sobre o bloco categórico *encoded*.
4. Concatenação: `[numérico_escalado | cat_encoded | PCA_cat]`.
5. Treinamento com early stopping (300 rounds para CatBoost, 200 para LGB/XGB).

Este procedimento garante **zero data leakage** entre folds.

4.4 Nível 2 Meta-Learners

Quatro estratégias de blending são treinadas sobre as predições OOF, enriquecidas com **features de 2ª ordem** (média, desvio-padrão, máximo, mínimo e amplitude entre os 8 modelos):

Meta-Learner	OOF RMSE	Notas
RidgeCV	0.112582	Treinado sobre features de 2ª ordem
ElasticNetCV	0.112599	$\alpha=0.00007$, $\lambda_1=0.90$
LassoCV (+)	0.113698	<code>positive=True</code> ; pesos: <code>cat_c=0.35</code> , <code>cat_e=0.68</code>
Optuna (direto)	0.114212	Busca <i>simplex</i> sobre OOF base

4.5 Meta-Blending Final (Optuna)

Um segundo nível de Optuna combina os quatro meta-learners:

Pesos finais:

```
ridge      : 0.6167
elasticnet : 0.3831
lasso_pos  : 0.0001
direct_opt : 0.0001
```

OOF RMSE FINAL: 0.112577

4.6 Pseudo-Labeling (2ª Passagem)

Após o blending final, amostras de teste com baixa variância entre as predições dos 8 modelos (percentil 30 do desvio-padrão) são adicionadas ao conjunto de treino com suas labels previstas. O *pipeline* completo é re-treinado sobre esse conjunto aumentado, e as predições finais são extraídas da 2ª passagem.

```
std_test = test_matrix.std(axis=1)
confident = std_test < np.percentile(std_test, 30)
```

```
X_aug = pd.concat([X, X_test[confident]], ignore_index=True)
y_aug = pd.concat([y, pd.Series(final_log[confident])], ignore_index=True)
# re-treino completo do CV loop sobre X_aug, y_aug
```

5 Pré-processamento Inside-Fold

Para cada fold k:

```
X_train_k, X_val_k = split(X, fold=k)

imputer.fit(X_train_k[num])      X_train_num, X_val_num
scaler.fit(X_train_k[num])      (RobustScaler)

TargetEncoder.fit(X_train_k, y_train_k)
    X_train_te, X_val_te, X_test_te

PCA(10).fit(X_train_te)          X_train_pca, X_val_pca

X_full = hstack[num | te | pca]

Para cada base model m:
    model.fit(X_train_full, y_train, eval=(X_val_full, y_val))
    oof[val_idx, m] += model.predict(X_val_full)
    test_preds[m]   += model.predict(X_test_full) / n_folds
```

6 Resultados

6.1 RMSE por Base Learner (OOF, escala logarítmica)

Modelo	OOF RMSE
cat_e	0.114478
cat_c	0.115069
cat_a	0.115290
cat_b	0.116325
cat_d	0.117181
hgb	0.124021
lgb_best	0.124906
xgb_best	0.125658

6.2 Resultado Final

Métrica	Valor
OOF RMSE (log)	0.112577
Escala da predição	Preço em USD
Range de predições	[46 419 , 569 716]
Arquivo de submissão	n-rosenthal--submission-6.csv

7 Conclusão

O *pipeline* implementado alcançou OOF RMSE de **0.112577** em escala log, correspondendo a um erro médio de aproximadamente 12% no preço de venda.

Os principais fatores de sucesso foram:

1. **Engenharia de features cuidadosa:** 45 features derivadas de conhecimento do domínio imobiliário, com destaque para as interações `OverallQual` & `TotalSF` e os novos scores compostos (`ExterScore`, `FireScore`, `BsmtScore`).
2. **Correção de tipos antes da separação de colunas:** converter `MSSubClass`, `MoSold` e `YrSold` para `string` antes de definir `cat_cols`/~`num_cols` evita que variáveis nominais sejam tratadas como contínuas.
3. **Imputação semântica de `GarageYrBlt`:** usar `YearBuilt` em vez de 0 elimina um viés que confunde os modelos de boosting.
4. **Target encoding dentro do fold:** eliminou *data leakage* e permitiu capturar variações de preço por bairro (`Neighborhood`).
5. **Diversidade no ensemble:** combinar CatBoost (cinco variantes com configurações distintas), LGB, XGB e HGB gerou ganho consistente sobre qualquer modelo individual.
6. **Meta-blending via Optuna com features de 2ª ordem:** a adição de estatísticas (média, desvio, range) sobre as predições OOF enriqueceu o meta-learner; a combinação de Ridge e ElasticNet pesada pelo Optuna superou qualquer meta-learner isolado.
7. **Pseudo-labelling com re-treino completo:** a 2ª passagem do CV loop sobre o conjunto aumentado com amostras de teste de alta confiança contribuiu para redução adicional do erro.